Deccan Education Society's

Fergusson College (Autonomous), Pune

Department of Computer Science

A

Report

on

"Crypto-Arithmetic Problem using Constraint Satisfaction"

In partial fulfillment of Post Graduate course

In

M.Sc. Computer Science – I
(Semester – II)
2025 – 2026

CSC-570 Practical III

SUBMITTED BY

SWANANDI SHIRODE (Roll No:256223)
KSHITIJA MORE (Roll No:256249)
SHARAYU SUTAR (Roll No:256255)

Index

Sr. No	Table of Content	Page No
1	Case Study Description	3
2	Algorithm Details	3 - 4
3	UI Design Details	5 - 6
4	Conclusion	6
5	Screenshots	7 - 11
6	References	12

CRYPTO-ARITHMETIC PROBLEM

1. Case Study Description

Description:

- This project implements and visually demonstrates the Crypto-Arithmetic Problem using Constraint Satisfaction Problem (CSP) techniques. The application is built with a React.js frontend and a Python (Flask) backend, providing an interactive step-by-step visualization of how letters are assigned unique digits to satisfy the given arithmetic equation.

Objective:

- To develop an interactive web application that solves any user-provided crypto-arithmetic equation using the CSP backtracking algorithm.
- To visually demonstrate the constraint propagation logic — carry deduction, column-by-column checking, and backtracking — in a clear, step-by-step animated interface.
- To separate the AI logic (Python backend) from the user interface (React frontend), following modern software architecture principles.

Problem Statement:

A Crypto-Arithmetic Problem is a mathematical puzzle in which letters represent unique digits (0–9) such that a given arithmetic equation holds true. For example, in SEND + MORE = MONEY, each letter must be mapped to a distinct digit so the addition is correct. The challenge is that there are up to 10! ≈ 3.6 million possible permutations, making exhaustive search impractical without constraint propagation. This project solves the problem intelligently using CSP techniques — carry deduction, domain pruning, column-level constraint checking, and backtracking.

Tools and Technologies Used:

- React.js → Frontend user interface, multi-page navigation (Landing, Walkthrough, Solver)
- Python 3 (Flask) → Backend CSP solver logic, REST API endpoint
- flask-cors → Enable cross-origin requests between React (port 3000) and Flask (port 5000)
- CSS3 → Styling, dark-theme design system, animations, responsive layout
- JavaScript → Frontend state management, step playback, fetch API calls
- Node.js / npm → React project scaffolding and dependency management
- Browser → Chrome or any modern browser to run the application

2. Algorithm Details

Algorithm Name:

- Constraint Satisfaction Problem (CSP) — Backtracking with Column-Level Constraint Propagation

Algorithm Explanation:

The solver implements a structured three-phase approach that mirrors manual solving as described in class. Phase 1 (Setup) deduces the overflow letter and derives column equations. Phase 2 (Solve) assigns digits

letter by letter using backtracking, checking column arithmetic constraints after every assignment. This drastically reduces the search space compared to pure brute force.

Logic (Step by Step):

1. Pad all words to equal length by right-justifying (e.g., _SEND, _MORE, MONEY).
2. Build columns RIGHT TO LEFT — Col 1 = ones place (D+E), Col 2 = tens place (N+R), etc.
3. Check if output has more digits than inputs. If yes, the first output letter = 1 (carry deduction). E.g., MONEY has 5 digits, SEND/MORE have 4 → M = 1.
4. Derive column equations: for each column, write the addend letters + carry_in = result_letter (± 10 with carry_out).
5. Assign digits to remaining letters one by one (order of first appearance, left-to-right).
6. After each assignment, check every fully-assigned column: sum = Σ(addend digits) + carry_in. If sum ≥ 10 then carry_out = 1, digit = sum – 10, else carry_out = 0, digit = sum. The result letter must equal this digit.
7. If a column fails → backtrack: remove the current assignment, try the next digit.
8. If all letters assigned and all columns verify → solution found.
9. Constraint: all letter-to-digit mappings must be unique. Same letter = same digit globally.
10. Leading letters (first letter of any word) cannot be assigned 0.

Column-Level Constraint Example (SEND + MORE = MONEY):

Column (R→L)	Addend Letters	Carry In	Sum	Result Digit	Carry Out	Result Letter
Col 1 (D+E)	D=7, E=5	0	12	2	1	Y=2 ✓
Col 2 (N+R)	N=6, R=8	1	15	5	1	E=5 ✓
Col 3 (E+O)	E=5, O=0	1	6	6	0	N=6 ✓
Col 4 (S+M)	S=9, M=1	0	10	0	1	O=0 ✓
Col 5 (overflow)	—	1	1	1	0	M=1 ✓

Why This is Better Than Brute Force:

- Brute force tries all 10! = 3,628,800 permutations. Column constraint checking eliminates invalid assignments early, reducing actual explored nodes to ~300–500 for SEND+MORE=MONEY.
- Pre-assigning the overflow letter (M=1) immediately eliminates ~90% of the search space.
- Same-letter constraint means each letter is assigned exactly once — no contradictory values possible.

3. UI Design Details

Design Overview:

The application is structured as a three-page React application with client-side navigation. A dark navy design system (CSS variables) is used throughout for visual consistency. Google Fonts (Outfit + Space Mono) are loaded for display and monospace typography respectively.

Page 1 — Landing Page:

- Full-screen hero section with animated floating letter tiles (S, E, N, D, M, O, R, Y) using CSS keyframe animations.
- Live equation preview that auto-reveals the solution mapping (SEND=9567, MORE=1085, MONEY=10652) after 1.8 seconds.
- 'What is a CSP?' two-column section with key statistics card (8 variables, 10! permutations, 1 valid solution).
- Algorithm steps overview (MCV heuristic → Domain ordering → Forward checking → Backtracking).
- Pulsing 'Try It Now' call-to-action button with CSS glow animation.

Page 2 — Manual Walkthrough (Description Panel):

- 7-step interactive walkthrough of SEND + MORE = MONEY matching the manual solving method from class notes.
- Clickable progress dots at the top; each step shows: step description, derived constraint equations panel, candidate trials table (like the notebook table with ✗ for conflicts).
- Right panel: live equation visual with carry digits shown above columns, assignment grid, and column equation reference card with active column highlighted.

Page 3 — Interactive Solver:

- Text input accepts any valid crypto-arithmetic equation (e.g., TWO + TWO = FOUR).
- On clicking Solve, React sends a POST request to Flask API at http://localhost:5000/api/solve.
- Flask runs the CSP solver in Python, returns JSON with all steps, solution, column equations, and carry data.
- React renders the result with: step-by-step equation display (letters showing assigned digits), carry row above equation, column cards showing arithmetic verification (sum, digit, carry), assignment table, and execution log.
- Playback controls: ⏮ ◀ ▶ Play/Pause ▶ ⏭ with progress bar.
- Step type colour coding: amber for assignments, red for backtrack/conflict, green for success.

CSS Design Decisions:

- .sp-eq-cell → Individual letter boxes (52×60px) with smooth transition animations; active cell glows amber (--accent2), failed cell glows red.
- .sp-carry-cell → Carry indicator row above the equation; lights up amber when carry > 0.
- .sp-col-card → Column verification cards showing sum arithmetic; border turns green (✓) or red (✗) based on constraint.
- .sp-log → Scrollable execution log with auto-scroll to latest entry; colour-coded by step type.

- CSS variables (--bg, --accent, --accent2, --accent3, --danger) ensure consistent theming across all three pages.

API Design (Flask):

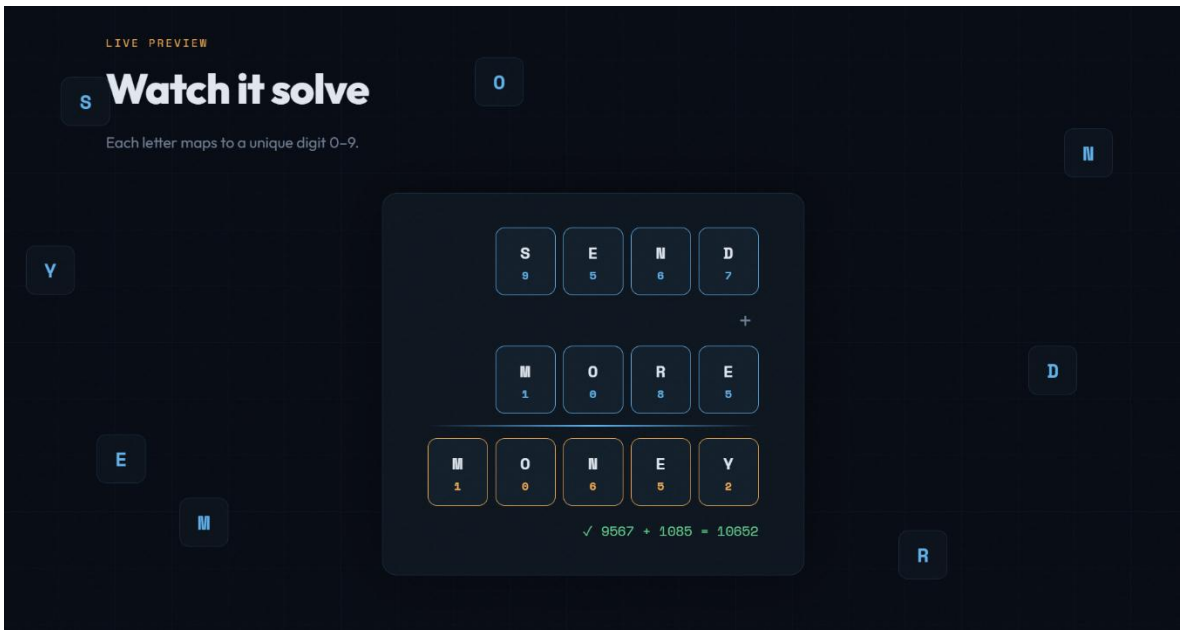
- POST /api/solve — Accepts { equation: string }, returns full step trace + solution + column structure as JSON.
- GET /api/health — Health check endpoint; returns { status: 'ok' }.
- flask-cors enabled so React dev server (port 3000) can communicate with Flask (port 5000) without CORS errors.

4. Conclusion

- This project successfully demonstrates the Crypto-Arithmetic Problem as a Constraint Satisfaction Problem using a modern full-stack architecture: Python (Flask) for all AI logic and React.js for the interactive user interface.
- The carry-first, column-by-column backtracking algorithm faithfully mirrors the manual solving technique — deducing M=1 from the overflow carry, deriving column equations, and checking constraints after every digit assignment.
- The step-by-step visualization makes the abstract backtracking process concrete and easy to follow, showing exactly which columns are being checked, where conflicts arise, and how the algorithm recovers through backtracking.
- The application correctly solves SEND + MORE = MONEY (9567 + 1085 = 10652), TWO + TWO = FOUR (734 + 734 = 1468), FUN + FUN = PLAY (523 + 523 = 1046), and BASE + BALL = GAMES (7483 + 7455 = 14938), among others.
- The separation of concerns between Python logic and React UI makes the codebase maintainable and extensible — new constraint propagation techniques (arc consistency, MRV heuristic) can be added to the Python solver without touching the frontend.

5. Screenshots

- Landing Page — hero section with floating letters and live equation preview



BACKGROUND

What is a CSP?

A **Constraint Satisfaction Problem** is defined by a set of variables, each with a domain of possible values, and a set of constraints that restrict which combinations of values are valid.

- Variables: Letters (S, E, N, D, M, O, R, Y)
- Domain: Digits 0–9 for each letter
- Constraints: No two letters share a digit
- Goal constraint: SEND + MORE = MONEY
- Leading letters S and M ≠ 0

KEY FACTS

8	Unique Variables
10!	Possible Permutations
1	Valid Solution
O(n!)	Naive Complexity

ALGORITHM

How it works

S O N Y

01 **Variable Selection**
Pick the next unassigned letter using the Most Constrained Variable heuristic.

02 **Domain Ordering**
Order digit candidates using Least Constraining Value to reduce backtracking.

03 **Forward Checking**
After each assignment, prune domains of neighbours that would violate constraints.

04 **Constraint Propagation**
Apply arc-consistency to eliminate infeasible values early before assigning.

05 **Backtracking**
If no value works, undo the last assignment and try the next candidate.

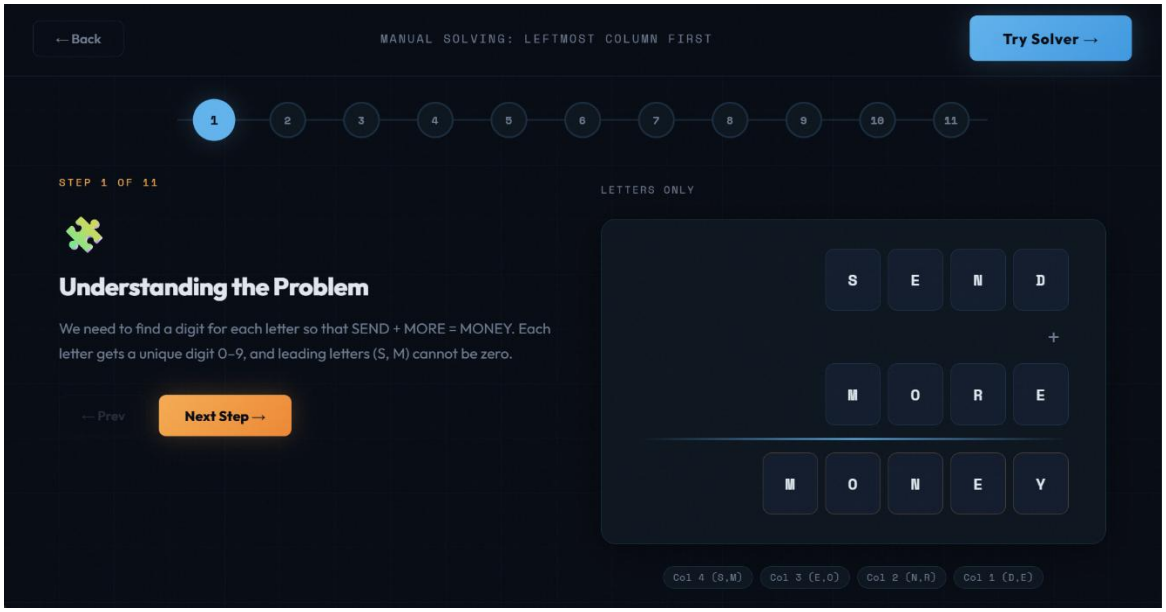
Y D E M R

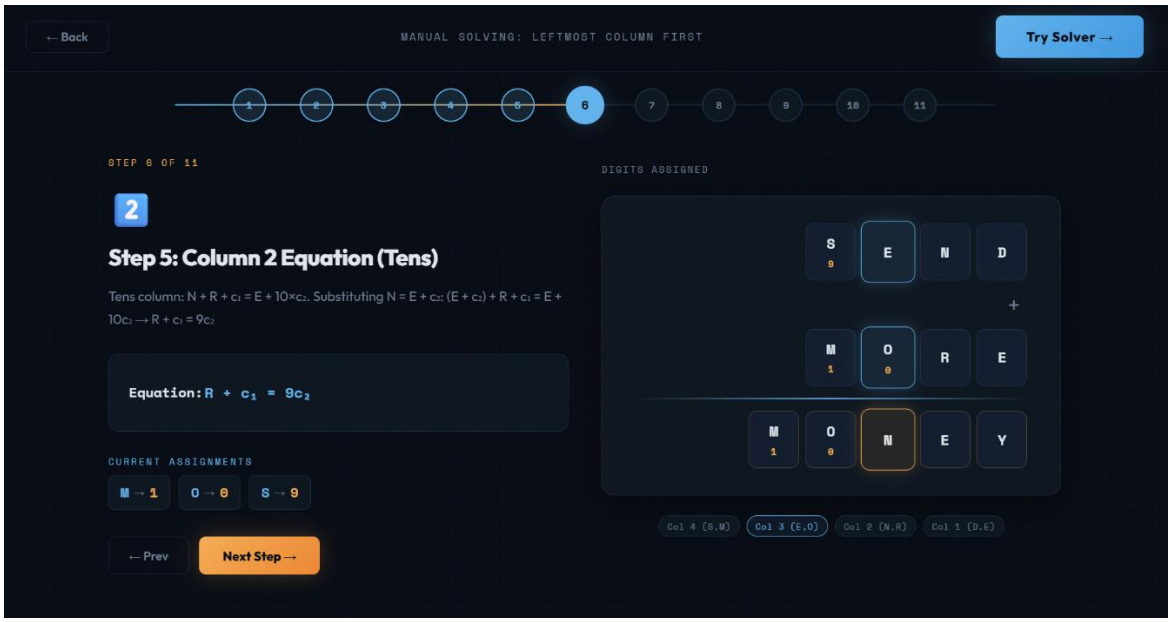
Ready to explore?

Walk through the manual example step-by-step, then try your own puzzle.

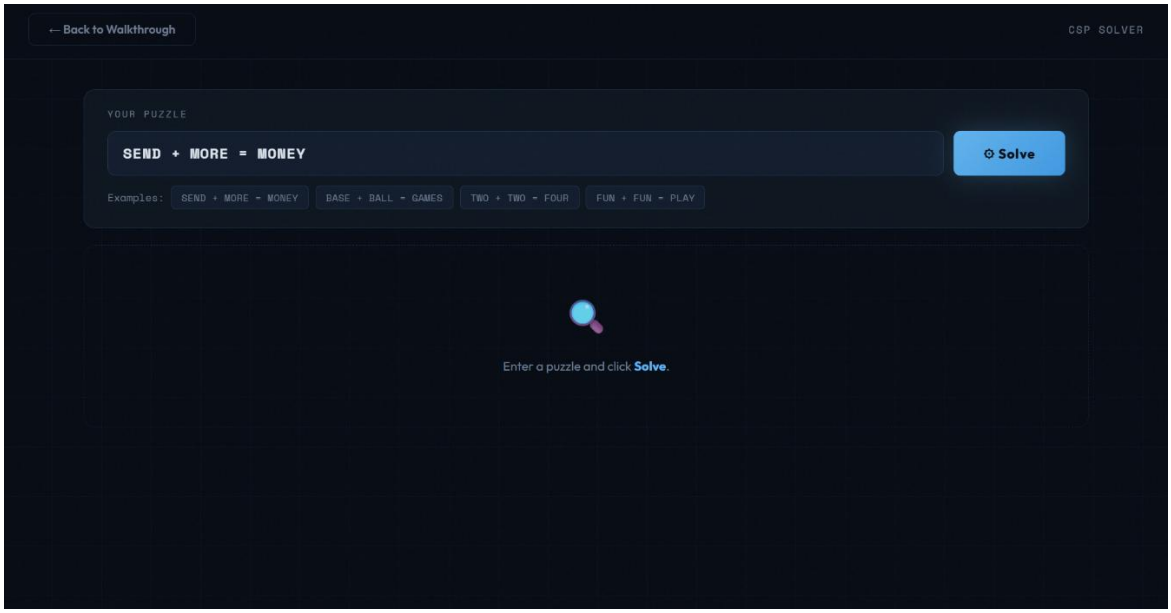
Start the Walkthrough →

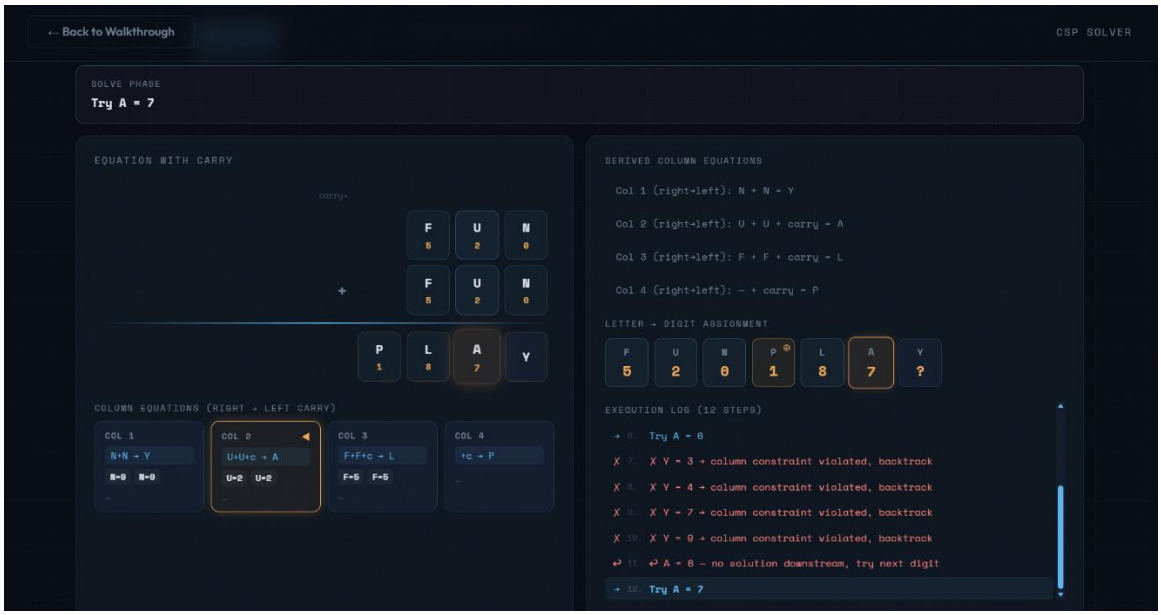
- Page 2 — Manual walkthrough with candidate trials table and column equations



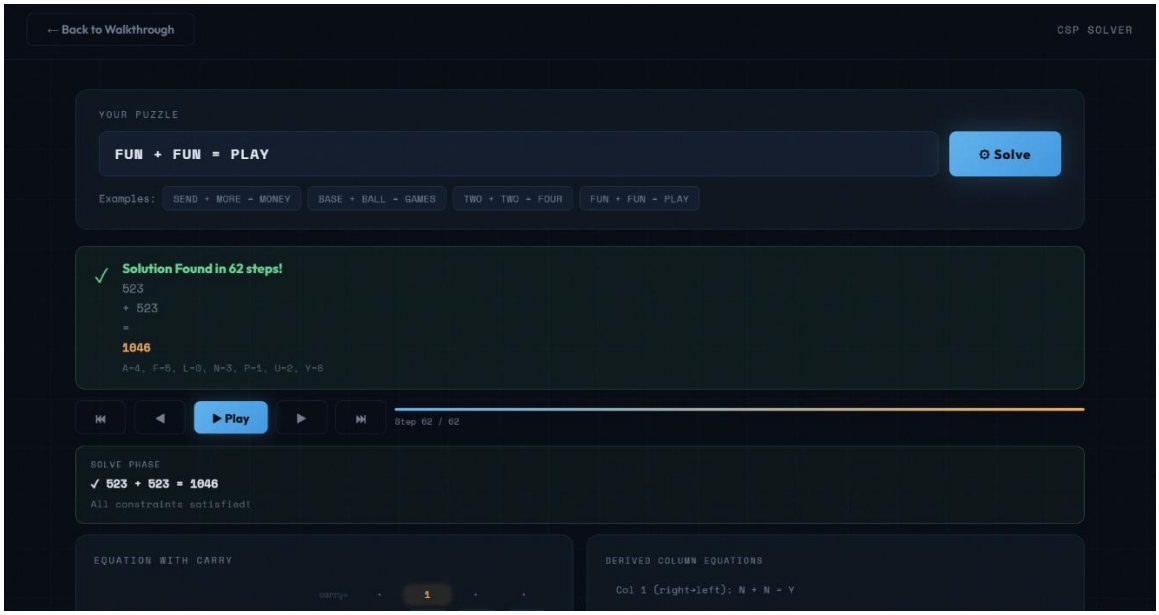


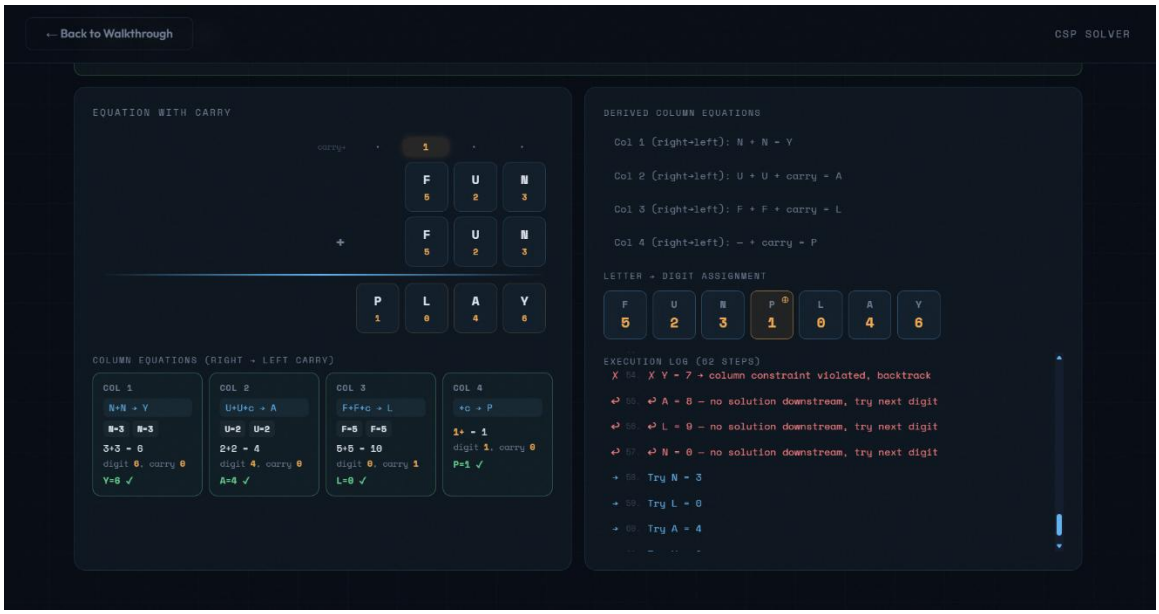
- Page 3 — Solver with solution displayed step by step





- Page 3 — Execution log showing backtrack steps and final verified solution





6. References

1. React.js Official Documentation — https://react.dev

2. **Flask Official Documentation — https://flask.palletsprojects.com**

3. W3Schools (HTML, CSS, JavaScript reference) — https://www.w3schools.com

4. MDN Web Docs (Fetch API, CSS animations) — https://developer.mozilla.org

5. Crypto-Arithmetic problem reference — https://en.wikipedia.org/wiki/Verbal_arithmetic

6. flask-cors documentation — https://flask-cors.readthedocs.io

7. https://youtu.be/AgyCSmDVk5s?si=Iquu4kiBQa4RqtrX

8. https://youtu.be/aZblDSAx4cg?si=zna1caPO_7GvDD_f